Architecture Documentation

2026-02-02

Contents

1 JAI-Arden: Domain-Specific Language Interpreter ... 5
1.1 System Overview ... 5
1.1.1 Key Objectives ... 5
1.2 Key Features ... 5
1.2.1 1. **Temporal Data Support** ... 5
1.2.2 2. **Expression Language** ... 5
1.2.3 3. **Data Structures** ... 5
1.2.4 4. **Built-in Functions** ... 5
1.2.5 5. **Control Flow** ... 5
1.3 External Dependencies ... 6
1.3.1 Runtime Dependencies ... 6
1.4 Compilation & Execution Flow ... 6
1.5 Data Flow ... 7
1.6 File Organization ... 7
1.7 Technology Stack ... 7
1.8 Key Design Patterns ... 8
1.8.1 1. **Pipeline Architecture** ... 8
1.8.2 2. **Value-Based Semantics** ... 8
1.8.3 3. **Broadcasting/Element-Wise Operations** ... 8
1.8.4 4. **Lazy Evaluation with Environments** ... 8
1.8.5 5. **Time Tag Preservation** ... 8
1.9 Next Steps ... 8
2 Components & Modules ... 9
2.1 High-Level Architecture ... 9
2.2 Component 2: Parser (`parser.jai`) ... 9
2.2.1 Architecture ... 9
2.2.2 Grammar Highlights ... 10
2.2.3 AST Node Types ... 10
2.2.4 Output Example ... 10
2.2.5 Main Functions ... 11
2.3 Component 3: Interpreter (`interpreter.jai`) ... 11
2.3.1 Execution Model ... 11
2.3.2 Value System ... 12
2.3.3 Execution Types ... 12
2.3.4 Operation Categories ... 12
2.3.5 Broadcasting Rules ... 13
2.3.6 Main Functions ... 14
2.3.7 Evaluation Example ... 14
2.4 Component 4: Main Entry Point (`main.jai`) ... 14
2.4.1 Control Flow ... 14
2.4.2 File Handling ... 15
2.5 Component Interactions ... 15
2.6 Summary Table ... 16
2.7 Next Steps ... 16
3 Data Flow & Evaluation Patterns ... 17
3.1 Complete Request Processing Flow ... 17
3.2 WHERE Filtering Pattern ... 17
3.3 Aggregation Pipeline ... 18

3.3.1 Maximum Operation ... 18
3.3.2 Temporal Aggregation ... 18
3.4 Control Flow Execution ... 19
3.4.1 IF Statement ... 19
3.4.2 FOR Loop ... 19
3.5 Type Coercion & Conversion ... 20
3.5.1 Implicit Conversions ... 20
3.5.2 Type Checking ... 20
3.6 Duration Calculation Pipeline ... 20
3.6.1 Simple Duration ... 20
3.6.2 Duration Arithmetic ... 21
3.7 Variable Binding & Lookup ... 21
3.7.1 Environment Stack ... 21
3.7.2 Lookup Flow ... 21
3.7.3 Assignment Flow ... 22
3.8 CSV Data Loading ... 22
3.8.1 READ Operator ... 22
3.9 Error Handling Flow ... 23
3.9.1 Error Cases ... 23
3.10 Summary: Data Transformation Pipeline ... 23
4 Interpreter Module Deep Dive ... 25
4.1 Overview ... 25
4.2 Core Architecture ... 25
4.3 Value Type System ... 25
4.3.1 Core Value Structure ... 25
4.3.2 Type Classification ... 26
4.3.3 Time Tagging ... 26
4.4 Execution Types ... 27
4.4.1 Broadcasting Rules ... 27
4.5 Operation Handler Categories ... 28
4.5.1 Binary Arithmetic (ElementWise) ... 28
4.5.2 Binary Comparison (ElementWise) ... 28
4.5.3 Unary Functions (ElementWise) ... 28
4.5.4 Unary Aggregations (ListWise) ... 29
4.5.5 Duration Operations ... 29
4.5.6 Special Operations ... 30
4.6 Control Flow Patterns ... 31
4.6.1 Evaluation Order ... 31
4.7 Variable Environment ... 31
4.7.1 Implementation ... 31
4.7.2 Scope ... 32
4.8 Type Conversion & Coercion ... 32
4.8.1 Implicit Conversions ... 32
4.9 Temporal Operations ... 32
4.9.1 Time Arithmetic ... 32
4.9.2 Temporal Aggregation ... 33
4.9.3 Occurrence Operators ... 33
4.10 Error Handling ... 33
4.10.1 Runtime Errors ... 33
4.10.2 Type Checking at Runtime ... 33
4.11 Performance Characteristics ... 34

4.12 Integration with Parser ... 34
4.13 Key Functions ... 34
4.14 Summary ... 34
5 Tokenizer Module Deep Dive ... 35
5.1 Overview ... 35
5.2 Architecture ... 35
5.3 Data Structures ... 36
5.3.1 Token Type ... 36
5.3.2 Token_Type Enum (40+ variants) ... 36
5.4 Scanning Algorithm ... 36
5.4.1 Main Loop ... 36
5.4.2 Character Classification ... 37
5.5 Recognition Patterns ... 37
5.5.1 1. Identifiers & Keywords ... 37
5.5.2 2. Numbers ... 37
5.5.3 3. Quoted Strings ... 38
5.5.4 4. ISO 8601 Dates & Times ... 38
5.5.5 5. Multi-Character Operators ... 38
5.6 Position Tracking ... 39
5.6.1 Line & Column Management ... 39
5.6.2 Token Position Example ... 39
5.7 Output: JSON Serialization ... 39
5.7.1 Process ... 39
5.7.2 Example Output ... 40
5.8 Comments ... 40
5.8.1 Single-Line Comments ... 40
5.9 Error Handling ... 41
5.9.1 Invalid Characters ... 41
5.9.2 Unterminated Strings ... 41
5.10 Performance Characteristics ... 41
5.11 Integration Points ... 41
5.12 Testing ... 42
5.12.1 Test Patterns ... 42
5.13 Key Takeaways ... 42

1 JAI-Arden: Domain-Specific Language Interpreter

1.1 System Overview

JAI-Arden is a complete domain-specific language (DSL) implementation written in JAI, designed to process time-based temporal query language with advanced data analytics capabilities. The system implements a full compiler pipeline: tokenization → parsing → interpretation, enabling users to write expressive queries over temporal data.

1.1.1 Key Objectives

- **Temporal Data Processing**: Support time-aware expressions and aggregations on time-series data
- **Type Safety**: Implement strong typing with support for Numbers, Strings, Booleans, Times, and Durations
- **Data Transformation**: Enable complex queries including filtering, aggregation, and mathematical operations
- **CSV Integration**: Read and process data from CSV files with timestamp metadata
- **List Operations**: Broadcasting and element-wise operations on collections

1.2 Key Features

1.2.1 1. Temporal Data Support

- Native time literals (ISO 8601: `YYYY-MM-DDTHH:MM:SS`)
- Duration operations (years, months, weeks, days, hours, minutes, seconds)
- Time arithmetic and comparisons
- Occurrence operators for event timing

1.2.2 2. Expression Language

- Arithmetic operations: `+`, `-`, `*`, `/`, `**` (power)
- Comparison operators: `<`, `>`, `<=`, `>=`, `=`, `<>`
- Logical operations: `IS`, `IS NOT`, `WHERE` (filtering)
- Temporal operators: `BEFORE`, `IS BEFORE`, `OCCUR`

1.2.3 3. Data Structures

- **Scalars**: Numbers, Strings, Booleans, Timestamps, Durations
- **Lists**: Homogeneous collections with element-wise operations
- **Values**: First-class values with optional "primary time" attribute

1.2.4 4. Built-in Functions

- **Aggregations**: `MAXIMUM`, `MINIMUM`, `AVERAGE`, `COUNT`, `FIRST`, `LATEST`, `EARLIEST`
- **Transformations**: `UPPERCASE`, `SQRT`, `TIME`, `INTERVAL`, `INCREASE`
- **Type Checks**: `IS NUMBER`, `IS LIST`, `IS NOT NULL`
- **Analytics**: `RANGE`, `WITHIN`, `ANY`

1.2.5 5. Control Flow

- Conditional statements: `IF ... THEN ... ELSE ... ENDIF`
- Loops: `FOR ... IN ... DO ... ENDDO`
- Statement blocks with proper scope

1.3 External Dependencies

```
graph LR
    JAI["JAI Language<br/>Standard Library"]
    CJSON["cJSON Library<br/>JSON Parsing"]
    LEMON["Lemon Parser<br/>Generator"]
    LIBC["libc<br/>Memory Management"]

    JaiArden["JAI-Arden"]

    JAI -->|import Basic, File| JaiArden
    CJSON -->|JSON parsing| JaiArden
    LEMON -->|Parser Grammar| JaiArden
    LIBC -->|c_free()| JaiArden
```

1.3.1 Runtime Dependencies

- **JAI Standard Library**: Basic I/O, File operations, Memory management
- **cJSON**: JSON parsing and manipulation
- **Lemon Parser Generator**: Grammar-based parser compilation
- **libc**: Memory allocation/deallocation

1.4 Compilation & Execution Flow

```
graph TD
    Source["Source Code<br/>program.arden"]

    Compile["JAI Compiler<br/>jai-linux"]

    subgraph "Build Artifacts"
        Binary["Binary<br/>main"]
    end

    Execute["./main<br/>program.arden"]

    Tokenize["Tokenization"]
    ParseStep["Parsing"]
    InterpStep["Interpretation"]

    Output["Output<br/>stdout"]

    Source -->|Source Files| Compile
    Compile -->|jai build| Binary

    Binary -->|Execution| Execute

    Execute -->|Input File| Tokenize
    Tokenize -->|Token Stream| ParseStep
    ParseStep -->|AST| InterpStep
    InterpStep -->|Evaluation| Output
```

1.5 Data Flow

```
graph LR
    Input["Input String<br/>DSL Program"]

    Tokens["Token Array<br/>[]Token"]
    JSON_STR["JSON String<br/>token list"]
    AST_JSON["cJSON* AST<br/>Parsed Tree"]

    EVAL["Evaluation<br/>Context"]
    ENV["Variable<br/>Environment"]

    RESULT["Result Value<br/>with optional Time"]

    Input -->|scan_tokens| Tokens
    Tokens -->|token_list_to_json| JSON_STR
    JSON_STR -->|cJSON_Parse| AST_JSON
    AST_JSON -->|interpret| EVAL

    EVAL -->|Lookup/Store| ENV
    EVAL -->|Compute| RESULT
```

1.6 File Organization

```
jai-arden/
├── main.jai              # Entry point & CLI
├── tokenizer.jai         # Lexical analysis
├── parser.jai            # Parser & AST generation
├── interpreter.jai       # Evaluation & execution
├── cjson/               # JSON parsing library
│   └── linux.jai
├── tests/               # Test files
├── docs/               # Documentation
├── glucose.csv         # Sample data
└── AGENTS.md           # Development guidelines
```

1.7 Technology Stack

Component	Technology	Purpose
Language	JAI	Systems programming language
Parser Generator	Lemon	Grammar-based LALR parser
JSON	cJSON	AST serialization/parsing
Build System	JAI Compiler	Native compilation
VCS	Jujitsu (jj)	Version control

1.8 Key Design Patterns

1.8.1 1. Pipeline Architecture

Sequential transformation: Source → Tokens → JSON → cJSON → Values

1.8.2 2. Value-Based Semantics

All computations return `Value` objects with optional time metadata:

```
Value {
  type_: Union[Number, String, Bool, Time, Duration, List, Unit]
  time: ?Float (optional primary time)
}
```

1.8.3 3. Broadcasting/Element-Wise Operations

Automatic type expansion for list operations: - Scalar ⊕ List → broadcast scalar, apply element-wise - List[n] ⊕ List[n] → zip and apply - List[n] ⊕ List[m] → error (length mismatch)

1.8.4 4. Lazy Evaluation with Environments

Variable binding through hash table, evaluated on demand during interpretation.

1.8.5 5. Time Tag Preservation

Operations preserve the "primary time" from left operand for temporal context.

1.9 Next Steps

See detailed architecture documentation in: - Components & Modules - Data Flow Patterns - Module deep dives in `modules/`

2 Components & Modules

2.1 High-Level Architecture

```
graph TB
    CLI["CLI Interface<br/>main.jai"]

    subgraph "Compiler Pipeline"
        direction LR
        TZ["Tokenizer<br/>tokenizer.jai"]
        PA["Parser<br/>parser.jai"]
        IN["Interpreter<br/>interpreter.jai"]
    end

    subgraph "External"
        CJSON["cJSON<br/>JSON Lib"]
        JAI["JAI Stdlib<br/>Basic, File"]
    end

    CLI -->|source code| TZ
    TZ -->|tokens| PA
    PA -->|AST| IN
    IN -->|output| CLI

    PA -.->|uses| CJSON
    IN -.->|uses| CJSON
    CLI -.->|uses| JAI
    TZ -.->|uses| JAI
```

2.2 Component 2: Parser (`parser.jai`)

Responsibility: Syntax analysis - convert tokens into Abstract Syntax Tree (AST)

2.2.1 Architecture

```
graph TD
    JSON_Input["JSON Token Stream<br/>from Tokenizer"]

    Lemon["Lemon Parser<br/>LALR(1) Grammar"]

    subgraph "Grammar Rules"
        Statements["Statement Block<br/>Programs"]
        Expressions["Expression Trees<br/>Operations"]
        Assignments["Assignments<br/>Variable Binding"]
        ControlFlow["Control Flow<br/>IF, FOR"]
    end

    ErrorHandler["Error Handler<br/>Syntax Errors"]

    cJSON_AST["cJSON* AST<br/>Parsed Program"]
```

```
    JSON_Input -->|Parse| Lemon
    Lemon -->|Rules| Statements
    Lemon -->|Rules| Expressions
    Lemon -->|Rules| Assignments
    Lemon -->|Rules| ControlFlow

    Lemon -->|Success| cJSON_AST
    Lemon -->|Error| ErrorHandler
    ErrorHandler -->|Report| cJSON_AST
```

2.2.2 Grammar Highlights

LALR(1) Parser with Precedence: 1. `WHERE` (lowest) 2. `AMPERSAND` (string concat) 3. Comparison operators: `<`, `>`, `<=`, `>=` 4. Occurrence operators: `ISWITHIN`, `ISBEFORE`, etc. 5. Arithmetic: `+`, `-` 6. Multiplication: `*`, `/` 7. Power: `**` (right-associative, highest)

2.2.3 AST Node Types

```
graph TD
    Node["AST Node<br/>JSON Object"]

    subgraph "Node Types"
        Literal["Literals<br/>NUMTOKEN, STRTOKEN, TIMETOKEN"]
        Unary["Unary Ops<br/>SQRT, UPPERCASE, NOT, TIME"]
        Binary["Binary Ops<br/>PLUS, MINUS, TIMES, IS BEFORE"]
        Ternary["Ternary Ops<br/>ISWITHIN, OCCURWITHIN"]
        Control["Control<br/>IF, FOR"]
        Assignment["Assignment<br/>Variable := Expression"]
    end

    Node -->|type field| Literal
    Node -->|type field| Unary
    Node -->|type field| Binary
    Node -->|type field| Ternary
    Node -->|type field| Control
    Node -->|type field| Assignment
```

2.2.4 Output Example

Input:

```
x := 42;
WRITE x + 10;
```

Output (cJSON AST):

```
{
  "type": "STATEMENTBLOCK",
  "statements": [
    {
      "type": "ASSIGN",
      "ident": "x",
      "arg": {"type": "NUMTOKEN", "value": "42"}
```

```
    },
    {
      "type": "WRITE",
      "arg": {
        "type": "PLUS",
        "arg": [
          {"type": "VARIABLE", "name": "x"},
          {"type": "NUMTOKEN", "value": "10"}
        ]
      }
    }
  ]
}
```

2.2.5 Main Functions

Function	Purpose
`parse_json()`	Parse JSON token stream into cJSON AST
Lemon parser	LALR(1) grammar compilation
Error handlers	Report syntax errors with line numbers

2.3 Component 3: Interpreter (`interpreter.jai`)

Responsibility: Semantic analysis and evaluation - execute AST and compute results

2.3.1 Execution Model

```
graph TB
    AST["AST Tree<br/>cJSON*"]

    Context["Execution Context<br/>Variables, State"]

    Evaluate["eval()<br/>AST Visitor"]

    subgraph "Operation Handlers"
        Arithmetic["Arithmetic<br/>op_plus, op_minus"]
        Logic["Logic<br/>op_is_before, op_lt"]
        Aggregation["Aggregation<br/>op_maximum, op_average"]
        Transform["Transform<br/>op_uppercase, op_sqrt"]
    end

    Value["Result Value<br/>type + optional time"]

    Output["Display Output<br/>print/write"]

    AST -->|Navigate| Evaluate
    Evaluate -->|Access| Context
    Evaluate -->|Dispatch| Arithmetic
    Evaluate -->|Dispatch| Logic
    Evaluate -->|Dispatch| Aggregation
    Evaluate -->|Dispatch| Transform
```

```
    Arithmetic -->|Produce| Value
    Logic -->|Produce| Value
    Aggregation -->|Produce| Value
    Transform -->|Produce| Value

    Value -->|Display| Output
```

2.3.2 Value System

```
Value :: struct {
    type_: Value_Union      // Actual value
    time: ?Float            // Optional primary time
}

Value_Union :: union {
    NumberLiteral: f64
    StringLiteral: string
    BoolLiteral: bool
    TimeLiteral: f64        // milliseconds since epoch
    DurationLiteral: f64    // milliseconds
    List: [dynamic]Value
    Unit                    // null
}
```

2.3.3 Execution Types

```
graph LR
    Operation["Operation"]

    ElementWise["ElementWise<br/>Applied per-element"]
    ListWise["ListWise<br/>Applied to whole list"]

    Scalar["Scalar<br/>x"]
    List["List<br/>[x, y, z]"]

    Operation -->|Mode| ElementWise
    Operation -->|Mode| ListWise

    ElementWise -->|Scalar ⊕ Scalar| ScalarResult["Result: Scalar"]
    ElementWise -->|List ⊕ List| ListResult["Result: List"]
    ElementWise -->|List ⊕ Scalar| ListResult

    ListWise -->|List| SingleResult["Result: Scalar or transformed"]
    ListWise -->|Scalar| SingleResult
```

2.3.4 Operation Categories

2.3.4.1 Binary Element-Wise Operations

- **Arithmetic**: `PLUS`, `MINUS`, `TIMES`, `DIVIDE`, `POWER`
- **Comparison**: `LT`, `ISGREATERT`
- **Temporal**: `BEFORE`, `ISBEFORE`, `ISNOTBEFORE`
- **Occurrence**: `OCCUREQUAL`, `OCCURBEFORE`, `OCCURAFTER`
- **String**: `AMPERSAND` (concatenation)

2.3.4.2 Unary Element-Wise Operations

- **Math**: `SQRT`, `UNMINUS`
- **Type Check**: `ISNUMBER`, `ISNOTNUMBER`, `TIME`
- **Duration**: `YEAR`, `MONTH`, `WEEK`, `DAY`, `HOURS`, `MINUTES`, `SECONDS`
- **Transform**: `UPPERCASE`

2.3.4.3 Unary List-Wise Operations

- **Aggregation**: `MAXIMUM`, `MINIMUM`, `AVERAGE`, `COUNT`
- **Selection**: `FIRST`, `LATEST`, `EARLIEST`, `ANY`
- **Analysis**: `INCREASE` (differences), `INTERVAL` (time gaps)

2.3.4.4 Ternary Operations

- **Range**: `ISWITHIN`, `ISNOTWITHIN`
- **Occurrence**: `OCCURWITHIN`

2.3.4.5 Special Operations

- **WHERE**: Filter with context variables `it`, `they`
- **ASSIGN**: Variable binding
- **IF/ELSE**: Conditional execution
- **FOR/IN**: Loop with iterator

2.3.5 Broadcasting Rules

```
graph TD
    A["Operand A<br/>Scalar or List"]
    B["Operand B<br/>Scalar or List"]

    Case1["Scalar ⊕ Scalar"]
    Case2["List[n] ⊕ Scalar"]
    Case3["Scalar ⊕ List[n]"]
    Case4["List[n] ⊕ List[n]"]
    Case5["List[n] ⊕ List[m]<br/>n ≠ m"]

    Result1["Result: Scalar"]
    Result2["Result: List[n]<br/>scalar expanded"]
    Result3["Result: List[n]<br/>scalar expanded"]
    Result4["Result: List[n]<br/>element-wise"]
    Result5["Result: Unit<br/>error"]

    A --> Case1
    B --> Case1
    A --> Case2
    B --> Case2
    A --> Case3
    B --> Case3
    A --> Case4
    B --> Case4
    A --> Case5
    B --> Case5

    Case1 --> Result1
    Case2 --> Result2
    Case3 --> Result3
```

```
    Case4 --> Result4
    Case5 --> Result5
```

2.3.6 Main Functions

Function	Signature	Purpose
`interpret()`	`(ast: cJSON*) -> unit`	Main evaluation entry point
`eval()`	`(ctx: *Context, node: cJSON*) -> Value`	Recursive AST evaluation
`unary_operation()`	Execute unary operations with broadcasting	
`binary_operation()`	Execute binary operations with broadcasting	
`ternary_operation()`	Execute ternary operations with broadcasting	
`where_filter()`	Filter with context variables	

2.3.7 Evaluation Example

```
Program: x := [1, 2, 3]; WRITE x + 10;

1. Tokenize:     [IDENTIFIER(x), ASSIGN, LSPAR, NUMTOKEN(1), ...]
2. Parse:        ASSIGN{ident:x, arg:LIST{items:[1,2,3]}}
                 WRITE{arg:PLUS{left:VARIABLE(x), right:10}}
3. Interpret:
   - ASSIGN: env[x] = [1, 2, 3]
   - PLUS (ElementWise): [1+10, 2+10, 3+10] = [11, 12, 13]
   - WRITE: prints "[11., 12., 13.]"
```

2.4 Component 4: Main Entry Point (`main.jai`)

Responsibility: CLI interface, file I/O, orchestration

2.4.1 Control Flow

```
graph TD
    Start["main()"]

    Args["Get Command<br/>Line Arguments"]

    CheckArgs["args.count > 1?"]

    ReadFile["read_entire_file()<br/>Load DSL program"]

    Success["File Read OK?"]

    Tokenize["scan_tokens()<br/>Lex input"]

    ToJSON["token_list_to_json()<br/>Format tokens"]
```

```
    ParseAST["parse_json()<br/>Generate AST"]

    Interpret["interpret()<br/>Execute program"]

    Output["Print Results"]

    Exit["exit()"]

    Start --> Args
    Args --> CheckArgs

    CheckArgs -->|Yes| ReadFile
    CheckArgs -->|No| Exit

    ReadFile --> Success
    Success -->|No| Exit
    Success -->|Yes| Tokenize

    Tokenize --> ToJSON
    ToJSON --> ParseAST
    ParseAST --> Interpret
    Interpret --> Output

    style Exit fill:#ff6b6b
    style Output fill:#51cf66
```

2.4.2 File Handling

- **Input**: Read entire DSL program from file specified in CLI argument
- **Output**: Print tokens (JSON), AST (cJSON), and interpreted results to stdout
- **Memory**: Proper cleanup with `defer free()` and `c_free()`

2.5 Component Interactions

```
graph LR
    File["Input File<br/>.arden DSL"]

    Main["main()"]
    Tokens["Token Array"]
    JSON["JSON String"]
    AST["cJSON* Tree"]
    Context["Variable<br/>Environment"]
    Output["Stdout<br/>Results"]

    File -->|Read| Main
    Main -->|scan_tokens| Tokens
    Tokens -->|token_list_to_json| JSON
    JSON -->|cJSON_Parse| AST
    AST -->|eval| Context
    Context -->|Calculate| Output

    classDef module fill:#4a90e2,color:#fff
```

```
    classDef data fill:#50c878,color:#fff

    class Main,Tokens,JSON,AST,Context module
    class File,Output data
```

2.6 Summary Table

Component	File	Inputs	Outputs	Key Functions
Tokenizer	tokenizer.jai	String	Token[]	scan_tokens(), keyword matching
Parser	parser.jai	Token[]	cJSON*	parse_json(), Lemon grammar
Interpreter	interpreter.jai	cJSON*	Value	eval(), operations dispatch
CLI	main.jai	Arguments	File I/O	main(), orchestration

2.7 Next Steps

See: - Data Flow Patterns - Request/response flows - `modules/tokenizer.md` - Detailed tokenizer analysis - `modules/parser.md` - Grammar and AST details - `modules/interpreter.md` - Operation semantics

3 Data Flow & Evaluation Patterns

3.1 Complete Request Processing Flow

```
graph TD
    A["1. User Input<br/>DSL Program String"]
    B["2. Tokenization<br/>scan_tokens()"]
    C["3. Token Array<br/>[]Token"]
    D["4. JSON Conversion<br/>token_list_to_json()"]
    E["5. JSON String<br/>Token List in JSON"]
    F["6. JSON Parsing<br/>cJSON_Parse()"]
    G["7. cJSON Tree<br/>Abstract Syntax Tree"]
    H["8. Interpretation<br/>eval()"]
    I["9. Value Result<br/>With optional time"]
    J["10. Output<br/>Formatted display"]

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> H
    H --> I
    I --> J

    style A fill:#e8f4f8
    style J fill:#f0f8e8
    style B fill:#fff4e8
    style D fill:#fff4e8
    style F fill:#fff4e8
    style H fill:#f4e8ff
```

3.2 WHERE Filtering Pattern

The `WHERE` operator implements context-aware filtering.

```
Input: [10, 20, 30] where it > 15

1. Parse:
   WHERE {
     left: LIST{[10, 20, 30]},
     right: GT{
       left: VARIABLE(it),
       right: NUMTOKEN(15)
     }
   }

2. Interpret:
   a. Evaluate left: [10, 20, 30]
```

```
   b. For each element:
      - Set "it" = current element
      - Evaluate right expression with "it" context
      - Collect elements where predicate is true

   Iteration:
   - it=10: eval(10 > 15) → false, exclude
   - it=20: eval(20 > 15) → true, include
   - it=30: eval(30 > 15) → true, include

   c. Collect results: [20, 30]

Output: [20., 30.]
```

3.3 Aggregation Pipeline

3.3.1 Maximum Operation

```
Input: maximum [100, 200, 150]

1. Parse:
   MAXIMUM {
     arg: LIST{items: [100, 200, 150]}
   }

2. Interpret (ListWise mode):
   a. Evaluate LIST → Value{List([100, 200, 150]), None}
   b. Extract numeric values: [100.0, 200.0, 150.0]
   c. Apply max function: 200.0
   d. Wrap result: Value{NumberLiteral(200.0), None}

Output: 200.
```

3.3.2 Temporal Aggregation

```
Input: average [time1, time2, time3]

Where time1, time2, time3 are values with time fields.

1. Parse:
   AVERAGE {
     arg: LIST{items: [...]}
   }

2. Interpret:
   a. Evaluate each item → [Value, Value, Value]
   b. Extract time fields: [t1, t2, t3] (milliseconds)
   c. Compute average: (t1 + t2 + t3) / 3 = t_avg
   d. Return as TimeLiteral: Value{TimeLiteral(t_avg), Some(t_avg)}

Output: ISO 8601 formatted timestamp
```

3.4 Control Flow Execution

3.4.1 IF Statement

```
Input: IF x < 5 THEN WRITE "small"; ELSE WRITE "large"; ENDIF;

1. Parse:
   IF {
     condition: LT{VARIABLE(x), NUMTOKEN(5)},
     thenbranch: WRITE{STRTOKEN("small")},
     elsebranch: WRITE{STRTOKEN("large")}
   }

2. Interpret:
   a. Evaluate condition
   b. If BoolLiteral(true): execute thenbranch
      - eval(WRITE{STRTOKEN("small")})
      - Output: "small"
   c. Else: execute elsebranch
      - eval(WRITE{STRTOKEN("large")})
      - Output: "large"
```

3.4.2 FOR Loop

```
Input: FOR i IN [1, 2, 3] DO WRITE i; ENDDO;

1. Parse:
   FOR {
     varname: "i",
     expression: LIST{[1, 2, 3]},
     statements: WRITE{VARIABLE(i)}
   }

2. Interpret:
   a. Evaluate expression → [1.0, 2.0, 3.0]
   b. For each element:
      - Bind variable: env["i"] = element
      - Execute statements
      - Output element

   Iterations:
   - i=1: WRITE → outputs "1."
   - i=2: WRITE → outputs "2."
   - i=3: WRITE → outputs "3."
```

3.5 Type Coercion & Conversion

3.5.1 Implicit Conversions

```
Operation: "Value: " & 42

1. String concatenation (AMPERSAND operator)
2. Parse: AMPERSAND{
     left: STRTOKEN("Value: "),
     right: NUMTOKEN(42)
   }
3. Interpret:
   a. Left evaluates to StringLiteral("Value: ")
   b. Right evaluates to NumberLiteral(42)
   c. Convert right to string: "42."
   d. Concatenate: "Value: 42."

Output: Value: 42.
```

3.5.2 Type Checking

```
Input: trace [1, "hello", true] is number;

1. Parse:
   ISNUMBER {
     arg: LIST{[1, "hello", true]}
   }

2. Interpret (ElementWise):
   a. Evaluate list → [NumberLiteral(1), StringLiteral("hello"),
BoolLiteral(true)]
   b. Apply ISNUMBER element-wise:
      - NumberLiteral(1) → BoolLiteral(true)
      - StringLiteral("hello") → BoolLiteral(false)
      - BoolLiteral(true) → BoolLiteral(false)
   c. Collect: [true, false, false]

Output: [true, false, false]
```

3.6 Duration Calculation Pipeline

3.6.1 Simple Duration

```
Input: 2 days before 2025-11-04T15:21:00

1. Parse:
   BEFORE {
     left: DAYS{NUMTOKEN(2)},
     right: TIMETOKEN(2025-11-04T15:21:00)
   }
```

```
2. Interpret:
   a. eval(NUMTOKEN(2)) → NumberLiteral(2)
   b. eval(DAYS) → multiply by ms_per_day
      → DurationLiteral(2 * 86400000) = 172800000 ms
   c. eval(TIMETOKEN) → TimeLiteral(1730718060000)
   d. eval(BEFORE) → subtract duration from time
      → 1730718060000 - 172800000 = 1730545260000
   e. Wrap: TimeLiteral(1730545260000)

Output: 2025-11-02T15:21:00Z
```

3.6.2 Duration Arithmetic

```
Input: (time2 - time1) / 3600000

1. Subtract two times:
   → DurationLiteral(Δt in milliseconds)

2. Divide duration by milliseconds per hour:
   → DurationLiteral / NumberLiteral
   → Result is the duration in hours (as a number)

Example: 3600000 ms / 3600000 = 1.0 hour
```

3.7 Variable Binding & Lookup

3.7.1 Environment Stack

```
┌──────────────────────────────┐
│   Variable Environment       │
│   (HashMap[String → Value])  │
├──────────────────────────────┤
│ "x" → Value{Num(42), None}   │
│ "y" → Value{Str("hi"), None}│
│ "it" → Value{Num(5), Some(t)}   │ (temporary for WHERE)
│ "glucose" → Value{List[...], None} │
└──────────────────────────────┘
```

3.7.2 Lookup Flow

```
Input: WRITE x;

1. Parse:
   WRITE{arg: VARIABLE{name: "x"}}

2. Interpret:
   a. eval(VARIABLE{name: "x"})
   b. Lookup in environment: env["x"]
   c. If found: return Value
   d. If not found: return Unit (null)
```

```
   e. Execute WRITE: format_value(Value) → string
   f. Print to stdout

Output: (value of x)
```

3.7.3 Assignment Flow

```
Input: x := [1, 2, 3] + [10, 20, 30];

1. Parse:
   ASSIGN{
     ident: "x",
     arg: PLUS{left: LIST[...], right: LIST[...]}
   }

2. Interpret:
   a. Evaluate arg expression
   b. PLUS: element-wise addition
      → Value{List([11, 22, 33]), None}
   c. Store in environment: env["x"] = result
   d. Return Unit

3. Subsequent use:
   a. VARIABLE("x") lookup
   b. Return stored Value
```

3.8 CSV Data Loading

3.8.1 READ Operator

```
Input: x := READ "glucose.csv";

File format:
value,timestamp
81,2025-11-04T15:21:00
92,2025-11-04T18:24:00

1. Parse:
   ASSIGN{
     ident: "x",
     arg: READ{arg: STRTOKEN("glucose.csv")}
   }

2. Interpret:
   a. eval(READ)
   b. Open file: glucose.csv
   c. Parse each line:
      - Split by comma
      - Extract value and timestamp
      - Create Value{NumberLiteral(value), Some(timestamp_ms)}
   d. Collect into list
```

```
   e. Return Value{List([...]), None}

Result: x = [81←time1, 92←time2, ...]
        (values with primary times attached)
```

3.9 Error Handling Flow

```
graph TD
    A["Tokenization<br/>Error"]
    B["Parse Error<br/>Syntax"]
    C["Runtime Error<br/>Type mismatch"]
    D["Error<br/>Handler"]
    E["Error<br/>Message"]
    F["Stderr<br/>Output"]

    A --> D
    B --> D
    C --> D
    D --> E
    E --> F

    style D fill:#ff6b6b
    style F fill:#ff6b6b
```

3.9.1 Error Cases

1. **Tokenization**: Invalid characters, unterminated strings
2. **Parsing**: Syntax errors, unexpected tokens
3. **Runtime**: Type mismatches, undefined variables, division by zero
4. **Type System**: Incompatible operands (e.g., adding a string to a number)

3.10 Summary: Data Transformation Pipeline

```
Source Code
    ↓
[Tokenizer] → Tokens
    ↓
[JSON Formatter] → JSON String
    ↓
[cJSON Parser] → AST (cJSON*)
    ↓
[Interpreter] → Values
    ├─ Read Variables
    ├─ Evaluate Expressions
    ├─ Dispatch Operations
    ├─ Handle Control Flow
    └─ Format Output
    ↓
Console Output
```

Each stage transforms the representation, preserving semantics while preparing for the next stage.

4 Interpreter Module Deep Dive

4.1 Overview

The interpreter (`interpreter.jai`) executes the Abstract Syntax Tree (AST) produced by the parser. It implements semantic analysis, type checking, and value computation using a recursive visitor pattern with environment-based variable binding.

4.2 Core Architecture

```
graph TB
    AST["AST (cJSON*)"]

    Context["Execution Context<br/>Variables, State"]

    Evaluator["eval()<br/>Recursive Visitor"]

    subgraph "Operation Handlers"
        BinOps["Binary Operations<br/>50 variants"]
        UnOps["Unary Operations<br/>30 variants"]
        TernOps["Ternary Operations<br/>5 variants"]
        CtlFlow["Control Flow<br/>IF, FOR, WHILE"]
    end

    TypeSystem["Type System<br/>Value Union"]

    Output["Result Values<br/>Display/Store"]

    AST -->|Navigate| Evaluator
    Evaluator -->|Access| Context

    Evaluator -->|Dispatch| BinOps
    Evaluator -->|Dispatch| UnOps
    Evaluator -->|Dispatch| TernOps
    Evaluator -->|Dispatch| CtlFlow

    BinOps -->|Compute| TypeSystem
    UnOps -->|Compute| TypeSystem
    TernOps -->|Compute| TypeSystem
    CtlFlow -->|Compute| TypeSystem

    TypeSystem -->|Format| Output
```

4.3 Value Type System

4.3.1 Core Value Structure

```
Value :: struct {
    type_: Value_Union      // Actual value
    time: ?Float            // Optional primary time (ms since epoch)
```

```
}

Value_Union :: union {
    NumberLiteral: f64
    StringLiteral: string
    BoolLiteral: bool
    TimeLiteral: f64        // Milliseconds since epoch
    DurationLiteral: f64    // Milliseconds duration
    List: [dynamic]Value
    Unit                    // null/void
}
```

4.3.2 Type Classification

```
graph TD
    Value["Value"]

    Scalar["Scalar Types"]
    List["Collection"]

    Numeric["Numeric"]
    Temporal["Temporal"]
    Text["Text"]
    Logic["Logic"]
    Empty["Empty"]

    Value --> Scalar
    Value --> List

    Scalar --> Numeric
    Scalar --> Temporal
    Scalar --> Text
    Scalar --> Logic
    Scalar --> Empty

    Numeric --> Number["NumberLiteral<br/>f64"]
    Numeric --> Duration["DurationLiteral<br/>f64 (ms)"]

    Temporal --> Time["TimeLiteral<br/>f64 (ms since epoch)"]

    Text --> String["StringLiteral<br/>string"]

    Logic --> Bool["BoolLiteral<br/>true/false"]

    Empty --> Null["Unit<br/>null"]

    List --> ListVal["List<br/>[dynamic]Value"]
```

4.3.3 Time Tagging

Every value can have an optional **primary time** attribute, used for: - Temporal context in WHERE clauses - Time preservation across operations - “Time of” queries

```
Example:
  glucose := 95
  time glucose := 2025-11-04T15:21:00

  Value {
    type_: NumberLiteral(95),
    time: Some(1730718060000)  // milliseconds since epoch
  }
```

4.4 Execution Types

```
graph TB
    Operation["Operation"]

    Mode["Execution Mode"]

    ElementWise["ElementWise<br/>Apply per-element"]
    ListWise["ListWise<br/>Apply to collection"]

    Operation -->|Select| Mode
    Mode --> ElementWise
    Mode --> ListWise

    ElementWise -->|Broadcasting| ScalarScalar["Scalar ⊕ Scalar<br/>→ Scalar"]
    ElementWise -->|Broadcasting| ListScalar["List ⊕ Scalar<br/>→ List[n]"]
    ElementWise -->|Broadcasting| ScalarList["Scalar ⊕ List<br/>→ List[n]"]
    ElementWise -->|Broadcasting| ListList["List[n] ⊕ List[n]<br/>→ List[n]"]

    ListWise -->|Aggregation| SingleResult["→ Scalar<br/>or transformed"]
```

4.4.1 Broadcasting Rules

```
For ElementWise operations:

1. Scalar ⊕ Scalar
   → Apply operation directly
   → Result: Scalar

2. List[n] ⊕ Scalar
   → Broadcast scalar to List[n]
   → Apply element-wise
   → Result: List[n]

3. Scalar ⊕ List[n]
   → Broadcast scalar to List[n]
   → Apply element-wise
   → Result: List[n]

4. List[n] ⊕ List[n]
   → Zip and apply element-wise
   → Result: List[n]
```

```
5. List[n] ⊕ List[m] (n ≠ m)
   → Length mismatch error
   → Result: Unit (null)
```

4.5 Operation Handler Categories

4.5.1 Binary Arithmetic (ElementWise)

```
// Addition
op_plus :: (left: Value, right: Value) -> Value {
    switch (left.type_, right.type_) {
    case (NumberLiteral(l), NumberLiteral(r)):
        return Value{NumberLiteral(l + r), left.time}
    case (DurationLiteral(l), DurationLiteral(r)):
        return Value{DurationLiteral(l + r), left.time}
    case (TimeLiteral(t), DurationLiteral(d)):
        return Value{TimeLiteral(t + d), left.time}
    // ... more cases
    case:
        return UNIT
    }
}

// Similar for: MINUS, TIMES, DIVIDE, POWER
```

4.5.2 Binary Comparison (ElementWise)

```
Less Than (LT):
  case (NumberLiteral(l), NumberLiteral(r)):
    return BoolLiteral(l < r)

Is Before (ISBEFORE):
  case (TimeLiteral(t1), TimeLiteral(t2)):
    return BoolLiteral(t1 < t2)

Greater Than (ISGREATERT):
  case (NumberLiteral(l), NumberLiteral(r)):
    return BoolLiteral(l > r)
```

4.5.3 Unary Functions (ElementWise)

```
Examples:

SQRT:
  case NumberLiteral(n):
    return NumberLiteral(sqrt(n))

UPPERCASE:
  case StringLiteral(s):
    return StringLiteral(uppercase(s))
```

```
ISNUMBER:
  case NumberLiteral(_):
    return BoolLiteral(true)
  case _:
    return BoolLiteral(false)

UNMINUS:
  case NumberLiteral(n):
    return NumberLiteral(-n)
```

4.5.4 Unary Aggregations (ListWise)

```
MAXIMUM:
  case List(items):
    max_val := find_max(items)
    return NumberLiteral(max_val)
  case NumberLiteral(n):
    return NumberLiteral(n)

AVERAGE:
  case List(items):
    sum := sum_all(items)
    count := length(items)
    return NumberLiteral(sum / count)

COUNT:
  case List(items):
    return NumberLiteral(length(items))

FIRST:
  case List(items):
    return items[0]
  case scalar:
    return scalar
```

4.5.5 Duration Operations

```
Pattern: value UNIT -> DurationLiteral

Duration Units:
  2 YEARS    → 2 * 365 * 24 * 60 * 60 * 1000 ms
  3 MONTHS   → 3 * 30 * 24 * 60 * 60 * 1000 ms
  1 WEEK     → 1 * 7 * 24 * 60 * 60 * 1000 ms
  5 DAYS     → 5 * 24 * 60 * 60 * 1000 ms
  8 HOURS    → 8 * 60 * 60 * 1000 ms
  30 MINUTES → 30 * 60 * 1000 ms
  45 SECONDS → 45 * 1000 ms
```

4.5.6 Special Operations

4.5.6.1 WHERE (Filtering with Context)

```
eval_where :: (ctx: *Context, node: cJSON*) -> Value {
    left := eval(ctx, get_left(node))

    // Set context variable for filtering
    old_it := ctx.env["it"]
    defer ctx.env["it"] = old_it

    switch (left.type_) {
    case List(items):
        // Filter each item through predicate
        filtered: [dynamic]Value
        for item in items {
            ctx.env["it"] = item  // Bind context
            result := eval(ctx, get_right(node))
            if is_true(result) {
                append(&filtered, item)
            }
        }
        return Value{List(filtered), left.time}
    case scalar:
        // Single item: return if predicate is true
        ctx.env["it"] = scalar
        result := eval(ctx, get_right(node))
        return is_true(result) ? scalar : UNIT
    }
}
```

4.5.6.2 IF Statement

```
case "IF":
    condition := eval(ctx, get_condition(node))

    if is_true(condition) {
        return eval(ctx, get_then_branch(node))
    } else {
        return eval(ctx, get_else_branch(node))
    }
```

4.5.6.3 FOR Loop

```
case "FOR":
    varname := get_varname(node)
    iterable := eval(ctx, get_expression(node))

    switch (iterable.type_) {
    case List(items):
        for item in items {
            ctx.env[varname] = item  // Bind loop variable
            eval(ctx, get_statements(node))
        }
```

```
        return UNIT
    case:
        return UNIT
    }
```

4.6 Control Flow Patterns

4.6.1 Evaluation Order

```
Strict Evaluation (Eager):
  All sub-expressions evaluated before operation

Example:
  x := 10 + 5 * 2

  1. eval(5)        → 5.0
  2. eval(2)        → 2.0
  3. eval(5 * 2)    → 10.0
  4. eval(10)       → 10.0
  5. eval(10 + 10)  → 20.0

Short-Circuit (IF):
  Right branch not evaluated if condition false

Example:
  IF false THEN WRITE "yes"; ELSE WRITE "no"; ENDIF;

  1. eval(condition) → false
  2. Skip then branch
  3. eval(else branch)
```

4.7 Variable Environment

4.7.1 Implementation

```
Context :: struct {
    env: map[string]Value    // Variable bindings
    now: Float               // Current timestamp (ms)
}

// Lookup
lookup_variable :: (ctx: *Context, name: string) -> Value {
    return ctx.env.get(name) or UNIT
}

// Assignment
assign_variable :: (ctx: *Context, name: string, value: Value) {
    ctx.env[name] = value
}
```

4.7.2 Scope

Global Scope: All variables live in single hash table (flat scope)

Temporary Bindings: - `it`, `they` → Set during WHERE evaluation - Loop variables → Set during FOR loop - Restored after operation

```
Before WHERE:
  env = {x: 10, y: 20}

During WHERE [10, 20, 30] where it > 15:
  env = {x: 10, y: 20, it: 10}    (iteration 1)
  env = {x: 10, y: 20, it: 20}    (iteration 2)
  env = {x: 10, y: 20, it: 30}    (iteration 3)

After WHERE:
  env = {x: 10, y: 20}            (it removed)
```

4.8 Type Conversion & Coercion

4.8.1 Implicit Conversions

```
String Concatenation (AMPERSAND):
  "Value: " & 42
  → StringLiteral("Value: ") AMPERSAND NumberLiteral(42)
  → "Value: " + number_to_string(42)
  → StringLiteral("Value: 42.")

Type Checking:
  x is NUMBER
  → Check if x is NumberLiteral
  → Return BoolLiteral(true/false)

Time Extraction:
  time of x
  → If x has time field, return TimeLiteral
  → Else return UNIT
```

4.9 Temporal Operations

4.9.1 Time Arithmetic

```
2 days before 2025-11-04T15:21:00

1. Evaluate "2 days"
   → 2 * 86400000 = 172800000 ms (DurationLiteral)

2. Evaluate "2025-11-04T15:21:00"
   → 1730718060000 ms (TimeLiteral)

3. Apply BEFORE operator
   → Subtract: 1730718060000 - 172800000
```

```
   → Result: 1730545260000 (TimeLiteral)

4. Format output
   → ISO 8601: 2025-11-02T15:21:00Z
```

4.9.2 Temporal Aggregation

```
average [time1, time2, time3]

1. Collect all times with values attached
2. Extract time fields: [t1, t2, t3]
3. Compute average: (t1 + t2 + t3) / 3
4. Return as TimeLiteral with time field set
```

4.9.3 Occurrence Operators

```
OCCUREQUAL: primary_time(left) == time(right)
OCCURBEFORE: primary_time(left) < time(right)
OCCURAFTER: primary_time(left) > time(right)
OCCURWITHIN: time(left) ∈ [time(right1), time(right2)]
OCCURSAMEDAYAS: day_of(primary_time(left)) == day_of(primary_time(right))
```

4.10 Error Handling

4.10.1 Runtime Errors

```
Division by Zero:
  NumberLiteral(x) / NumberLiteral(0)
  → Return UNIT (null)

Length Mismatch:
  List[3] ⊕ List[5]
  → Return UNIT (null)

Type Mismatch:
  NumberLiteral(42) < StringLiteral("hello")
  → Return UNIT (null)

Undefined Variable:
  VARIABLE("undefined")
  → Return UNIT (null)
```

4.10.2 Type Checking at Runtime

All operations check types before computation:

```
case (NumberLiteral(l), NumberLiteral(r)):
    // Safe to compute
case (DurationLiteral(l), DurationLiteral(r)):
    // Safe to compute
case:
```

```
    // Incompatible types
    return UNIT
```

4.11 Performance Characteristics

Operation	Time	Space
Lookup variable	O(1)	Hash table
Arithmetic	O(1)	Constant
List operation	O(n)	List size
Aggregation	O(n)	List size
WHERE filter	O(n)	Result size

4.12 Integration with Parser

```
graph LR
    Parser["Parser<br/>cJSON* AST"]
    Interp["Interpreter<br/>eval()"]
    Output["Output Values"]

    Parser -->|cJSON*| Interp
    Interp -->|Value| Output
```

The interpreter takes a complete AST and recursively evaluates it, producing a final Value and outputting results.

4.13 Key Functions

Function	Purpose
`interpret()`	Entry point from main
`eval()`	Recursive AST visitor
`binary_operation()`	Dispatch binary ops with broadcasting
`unary_operation()`	Dispatch unary ops with broadcasting
`ternary_operation()`	Dispatch ternary ops with broadcasting
`where_filter()`	Filter with context variables
`value_to_string()`	Format values for output

4.14 Summary

The interpreter implements: 1. **Recursive evaluation** of AST nodes 2. **Type-safe operations** with runtime type checking 3. **Broadcasting semantics** for list operations 4. **Environment binding** for variable storage 5. **Control flow** with conditional and iterative statements 6. **Temporal operations** with time tagging and arithmetic 7. **Error resilience** returning UNIT on type mismatches

5 Tokenizer Module Deep Dive

5.1 Overview

The tokenizer (`tokenizer.jai`) performs lexical analysis—converting raw source code into a stream of meaningful tokens. It uses a state machine approach with lookahead to recognize keywords, operators, literals, and identifiers.

5.2 Architecture

```
graph TD
    Input["Raw Input String<br/>DSL Source Code"]

    Scanner["Scanner Loop<br/>Character by character"]

    subgraph "Recognition"
        KW["Keyword Matching<br/>IF, FOR, WRITE, etc."]
        OP["Operator Recognition<br/>+, -, :=, **, etc."]
        NUM["Number Parsing<br/>Integers & Floats"]
        STR["String Parsing<br/>Quoted text"]
        TIME["Time/Date Parsing<br/>ISO 8601"]
        ID["Identifier/Variable<br/>Custom names"]
    end

    TokenList["Token Array<br/>[]Token"]

    JSON_Output["JSON Output<br/>Token list as JSON"]

    Input --> Scanner
    Scanner --> KW
    Scanner --> OP
    Scanner --> NUM
    Scanner --> STR
    Scanner --> TIME
    Scanner --> ID

    KW --> TokenList
    OP --> TokenList
    NUM --> TokenList
    STR --> TokenList
    TIME --> TokenList
    ID --> TokenList

    TokenList --> JSON_Output
```

5.3 Data Structures

5.3.1 Token Type

```
Token :: struct {
    type: Token_Type
    literal: string
    line: int
    col: int
}
```

Provides: - **Type**: Token classification (keyword, operator, etc.) - **Literal**: Actual text from source (e.g., "42" or "hello") - **Position**: Line and column for error reporting

5.3.2 Token_Type Enum (40+ variants)

```
Keywords:        IF, FOR, THEN, ELSE, DO, ENDDO, WRITE, TRACE, READ
Types:           NUMBER, LIST, NUMBERTYPE, LISTTYPE
Operators:       PLUS, MINUS, TIMES, DIVIDE, POWER, LT, GT, LTEQ, GTEQ, EQ, NEQ
Logical:         IS, NOT, SAME, ANY, IN
Temporal:        TIME, DAY, HOUR, MINUTE, SECOND, YEAR, MONTH, WEEK
Aggregation:     MAXIMUM, MINIMUM, AVERAGE, COUNT, FIRST, LATEST, EARLIEST
Control:         INCREASE, INTERVAL, BEFORE, AFTER, WITHIN, OCCUR
Values:          NUMTOKEN, STRTOKEN, TIMETOKEN, TRUE, FALSE, NULL, NOW,
CURRENTTIME
Syntax:          LPAR, RPAR, LSPAR, RSPAR, COMMA, SEMICOLON, DOT, ASSIGN, RANGE,
AMPERSAND
```

5.4 Scanning Algorithm

5.4.1 Main Loop

```
scan_tokens :: (input: string, debug: bool) -> [dynamic]Token {
    tokens: [dynamic]Token
    i := 0
    line := 1
    col := 1

    while i < input.count {
        // Current character
        ch := input[i]

        // Multi-character lookahead for operators
        if i + 1 < input.count {
            two_char := input[i:i+2]
            if matches multi-char operator {
                create token, advance i by 2
                continue
            }
        }

        // Single character recognition
        if is_digit(ch) {
```

```
            parse_number()
        } else if is_letter(ch) {
            parse_identifier()
        } else if ch == '"' {
            parse_string()
        } else if is_operator(ch) {
            create operator token, advance i
        } else if is_whitespace(ch) {
            skip whitespace, track line/col
        }

        i += 1
    }

    return tokens
}
```

5.4.2 Character Classification

```
Whitespace:  ' ', '\t'
Newline:     '\n', '\r'
Digit:       '0'-'9'
Letter:      'a'-'z', 'A'-'Z', '_'
Operator:    '+', '-', '*', '/', '=', '<', '>', '!', etc.
String:      '\"'
```

5.5 Recognition Patterns

5.5.1 1. Identifiers & Keywords

```
Pattern: [a-zA-Z_][a-zA-Z0-9_]*

Examples:
  x           → IDENTIFIER
  myVar123    → IDENTIFIER
  IF          → keyword (case-insensitive)
  For         → keyword (FOR, case-insensitive)
  sqrt        → keyword (SQRT)

Process:
  a. Read alphanumeric/underscore sequence
  b. Check against keyword map
  c. If match: create keyword token
  d. Else: create IDENTIFIER token with name
```

5.5.2 2. Numbers

```
Pattern: [0-9]+ | [0-9]+ '.' [0-9]+

Examples:
  123         → NUMTOKEN(123.0)
```

```
  45.6        → NUMTOKEN(45.6)
  0           → NUMTOKEN(0.0)

Process:
  a. Read digits
  b. Check for decimal point
  c. If found, read fractional digits
  d. Convert to float
  e. Create NUMTOKEN with value
```

5.5.3 3. Quoted Strings

```
Pattern: '"' [^"]* '"'

Examples:
  "hello"                → STRTOKEN("hello")
  "multi\nline\nstring"  → STRTOKEN with newlines
  ""                     → STRTOKEN("")

Process:
  a. Skip opening quote
  b. Read until closing quote
  c. Track newlines for line counting
  d. Create STRTOKEN with string content
```

5.5.4 4. ISO 8601 Dates & Times

```
Date patterns:
  YYYY-MM-DD                    → TIMETOKEN(2025-11-04)
  YYYY-MM-DDTHH:MM:SS          → TIMETOKEN(2025-11-04T15:21:00)
  YYYY-MM-DDTHH:MM:SSZ         → TIMETOKEN(2025-11-04T15:21:00Z)

Time-only patterns:
  HH:MM                         → TIMETOKEN(15:21)
  HH:MM:SS                      → TIMETOKEN(15:21:00)

Parser:
  a. Detect digit → try date/time parsing
  b. Try YYYY-MM-DD pattern first
  c. If successful, check for T and time part
  d. Create TIMETOKEN with full string
```

5.5.5 5. Multi-Character Operators

```
Priority (checked in order):
  ":="      → ASSIGN
  "**"      → POWER
  "..."     → RANGE
  "<="      → LTEQ
  ">="      → GTEQ
  "<>"      → NEQ
  "//"      → COMMENT (skip to EOL)
```

```
  "*"       → TIMES
  "+"       → PLUS
  "-"       → MINUS
  "/"       → DIVIDE
  "="       → EQ
  "<"       → LT
  ">"       → GT
```

5.6 Position Tracking

5.6.1 Line & Column Management

```
// Initial state
line = 1, col = 1

// For each character:
if ch == '\n' {
    line += 1
    col = 1
} else if ch == '\r' {
    // Skip, but don't advance line (usually \r\n)
    col = 1
} else {
    col += 1
}

// String output includes line/col for error reporting
```

5.6.2 Token Position Example

```
Input:
  x := 42;
  WRITE x;

Tokens:
  Token { type = IDENTIFIER("x"), line = 1, col = 1 }
  Token { type = ASSIGN, line = 1, col = 3 }
  Token { type = NUMTOKEN(42), line = 1, col = 6 }
  Token { type = SEMICOLON, line = 1, col = 8 }
  Token { type = WRITE, line = 2, col = 1 }
  Token { type = IDENTIFIER("x"), line = 2, col = 7 }
  Token { type = SEMICOLON, line = 2, col = 8 }
```

5.7 Output: JSON Serialization

5.7.1 Process

```
token_list_to_json :: (tokens: [dynamic]Token) -> string {
    // Create JSON array
    result: [dynamic]u8
```

```
    append_string(&result, "[")

    // For each token
    for token, i in tokens {
        if i > 0 append_string(&result, ", ")

        // Create JSON array: [line, type, literal]
        append_string(&result, "[")
        append_int(&result, token.line)
        append_string(&result, ", \"")
        append_string(&result, token_type_to_string(token.type))
        append_string(&result, "\", \"")
        append_string(&result, escape_json(token.literal))
        append_string(&result, "\"]")
    }

    append_string(&result, "]")
    return to_string(result)
}
```

5.7.2 Example Output

Input:

```
x := 42;
WRITE x;
```

JSON Output:

```
[
  [1, "IDENTIFIER", "x"],
  [1, "ASSIGN", ":="],
  [1, "NUMTOKEN", "42"],
  [1, "SEMICOLON", ";"],
  [2, "WRITE", "WRITE"],
  [2, "IDENTIFIER", "x"],
  [2, "SEMICOLON", ";"]
]
```

5.8 Comments

5.8.1 Single-Line Comments

```
Pattern: "//" [^newline]*

Example:
  WRITE 42; // output the answer
  x := 1;   // initialize x

Processing:
  a. Detect "//"
```

```
  b. Skip all characters until newline
  c. Do not create tokens for comment text
  d. Continue scanning after newline
```

5.9 Error Handling

5.9.1 Invalid Characters

```
Unrecognized character → UNKNOWN token

Example:
  x @ y      → [IDENTIFIER(x), UNKNOWN(@), IDENTIFIER(y)]

Usage:
  Parser will report syntax error on UNKNOWN token
```

5.9.2 Unterminated Strings

```
Missing closing quote at EOF → Tokenizer reports error

Example:
  "hello world    (no closing quote)
  → Error: Unterminated string at line 1, col 1
```

5.10 Performance Characteristics

Operation	Time	Space
Scan N characters	O(N)	O(tokens)
Keyword lookup	O(1)	HashMap
Number parsing	O(digits)	Constant
String copying	O(len)	Per-token

5.11 Integration Points

```
graph LR
    Input["Input String"]
    TZ["Tokenizer"]
    Tokens["Token Array"]
    JSON["JSON String"]
    Parser["Parser"]

    Input -->|scan_tokens| TZ
    TZ -->|Output| Tokens
    Tokens -->|token_list_to_json| JSON
    JSON -->|Input| Parser
```

The tokenizer is the **first stage** of the compiler pipeline. Its output (JSON array of tokens) is fed directly to the parser.

5.12 Testing

5.12.1 Test Patterns

```
// Test simple operators
scan_tokens("+-*/;:=,()[]&<><=>=<>")
→ [PLUS, MINUS, TIMES, DIVIDE, SEMICOLON, ASSIGN, COMMA, ...]

// Test keywords
scan_tokens("If foo FOR bar")
→ [IF, IDENTIFIER(foo), FOR, IDENTIFIER(bar)]

// Test multi-line with comments
scan_tokens("WRITE 1; // comment\nWRITE 2;")
→ [WRITE, NUMTOKEN(1), SEMICOLON, WRITE, NUMTOKEN(2), SEMICOLON]

// Test time tokens
scan_tokens("2025-11-04T15:21:00")
→ [TIMETOKEN(2025-11-04T15:21:00)]

// Test strings with newlines
scan_tokens("\"hello\nworld\"")
→ [STRTOKEN(hello\nworld)]
```

5.13 Key Takeaways

1. **State Machine**: Character-by-character scanning with context
2. **Lookahead**: Peek ahead for multi-character operators
3. **Keyword Map**: Fast O(1) keyword recognition
4. **Position Tracking**: Accurate line/column for error messages
5. **Whitespace Handling**: Skips but tracks line numbers
6. **JSON Output**: Standardized token representation